UNIT 2: Linear Search, Merge Sort, Bubble Sort, Quick Sort, Binary Search, Strassen's Matrix, max min of array, Brut force, Insertion Sort, Convex Hull: finding Closet pair.

2.1 FINDING RECURRENCE RELATION FROM ALGORITHMS.

Q) A(n)
{ if (n>1) → T(1).
return A(n-1); → T(n-1)
}

Let the Time Complexity of function A(n) be T(n).
∴ when A(n) is called, T(n) would be its time complexity.
∴ when A(n-1) is called, T(n-1) would be its time complexity.
* for a basic 'if' statement, it would be '1', when n>1.
∴ Recurrance relation:

$$T(n) \begin{cases} T(n-1)+1 & ; n>1 \\ 1 & ; n \leq 1 \end{cases}$$

Q) func (int n) Let it be P(n)
{ if (n==1) ——→ 1
return;
else → 1
{ for (int i=1; i<n; i++)
for (int j=1; j<=n; j++) } n^2 } n^2
print ("*")
function (n-3); ——→ P(n-3)
}
∴ Recurrance Relation: $T(n) \begin{cases} T(n-3)+n^2 & , \text{otherwise} \\ 1 & , n=1. \end{cases}$

2.2 MERGE SORT.

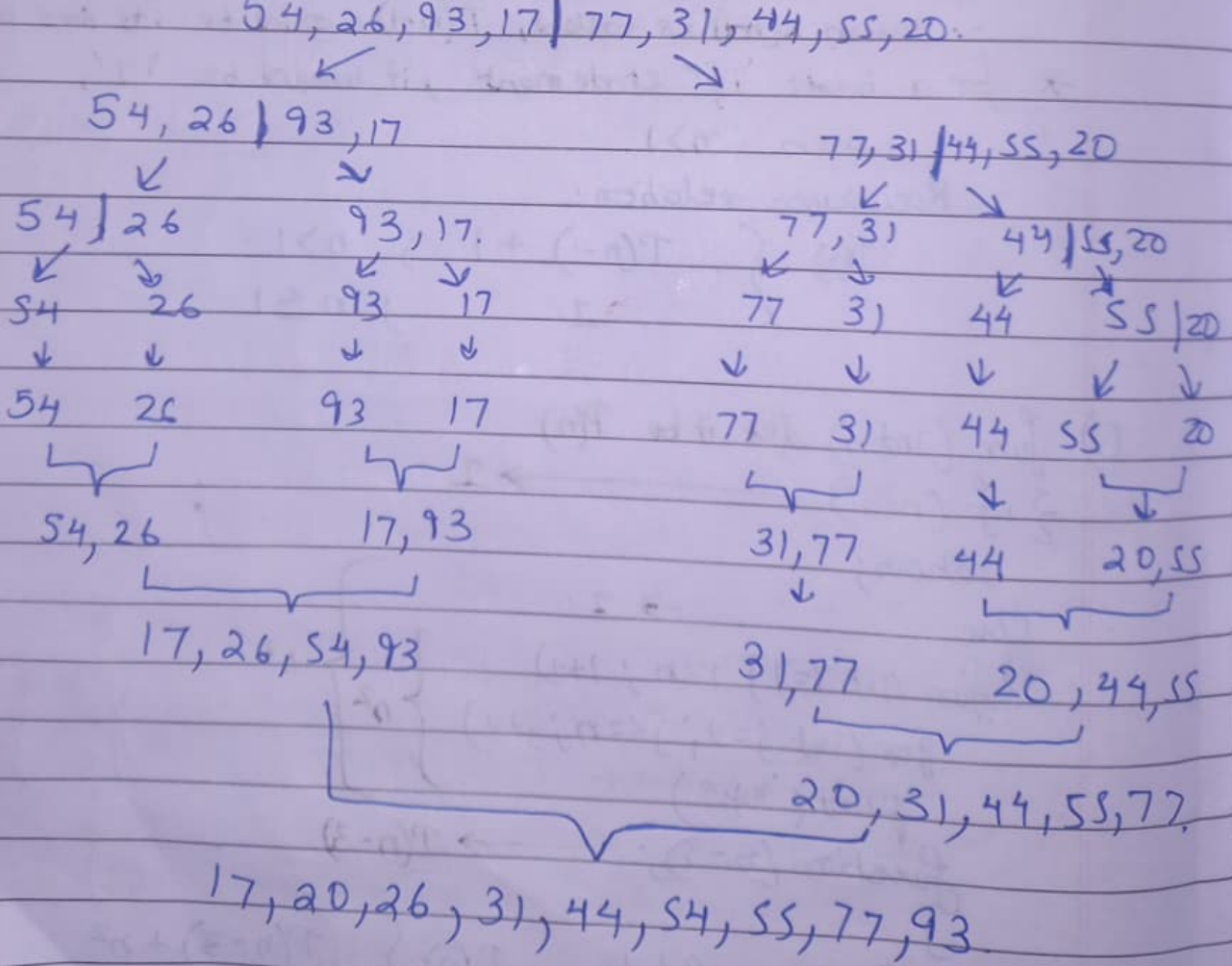
→ based on divide & conquer method.

→ The list given is to be sorted ; so the unsorted list is divided

→ Merge Sort works by recursively dividing the input array into two halves, recursively sorting the two halves and finally merging them back together to obtain the sorted array.

Given unsorted array: 54, 26, 93, 17, 77, 31, 44, 55, 20.

[Note : for odd number size, 1 more should be with the right side.]



Pseudocode:

```
function mergeSort(list)
    if length of list is 1 or less
        return list.
    split list into two halves: left and right.
    leftsorted = mergesort(left)
    rightsorted = mergesort(right)
    return merge(leftsorted, rightsorted)

function merge(left, right)
    create empty list called result.
    while both left and right have items.
        if first item in left is smaller than first in right
            move it to result.
        else
            move first item from right to result.
    add any remaining items from left to result.
    add any remaining items from right to result.
    return result.
```

TIME COMPLEXITY : $O(n \log n)$

Recurrance Relation:

$$T(n) = \begin{cases} 1 & , n = 1 \\ 2T(n/2) + O(n) & , n > 1. \end{cases}$$

2.3 BINARY SEARCH.

→ used to find a key in a sorted array.
→ the search space (array given) is split into 2 halves by finding the middle index "mid".
→ if key found in the middle, best, otherwise searched.

#ex: finding '7': in:

-5, -2, 0, 1, 2, 4, 5, 6, 7, 10.

here, middle: 2.

and 7 > 2 ; ∴ 7 is in the right side of 2:

then; 4, 5, 6, 7, 10.

here, middle: 6; 7 > 6 ; ∴ right side.

then; 7, 10

middle: 7 → found.

ALGORITHM:

Input: a sorted list and a target value
Output: The index of the target value, or -1 if not found

1. Set low = 0.
2. Set high = length of list - 1.
3. while low is less than equal to high:
   a. Set mid = (low + high) // 2.
   b. If list[mid] == target:
      - return mid
   c. If list[mid] is less than the target:
      - Set low = mid + 1
   d. else if list[mid] is more than target:
      - Set high = mid - 1
4. If the loop ends and the target was not found:
   - return -1.

TIME COMPLEXITY: Best: O(1)
As, worst: O(log n)

$T(n) = T(n/2) + 1$

2.4 Linear Search

Linearly searching a key in an array.

#Algorithm:

Input: A list and a target value.

output: the index of the target value, or -1 if not found.

1. Start from the first element of the list.
2. Repeat for each element of the list.
   a. If the current element = target:
      - return the current index.
3. If the end of the list is reached

TIME COMPLEXITY: Best: O(1)

Avg, worst: O(n).

2.5 Quick Sort.

① Choose a pivot: Select an element from the array as the pivot. The choice of the pivot can vary (eg, first, last, random, median).

② Partition the array: rearrange the array around the pivot. After partitioning, all elements smaller than the pivot will be on its left, and all elements greater than the pivot will be on its right. The pivot is then it its correct position.

pivot → items in left are smaller.
→ items in right are larger.

Ex 1) 2, 6, 5, 3, 8, 7, 1, 0.

Let us select '3' as pivot.

⇒ We take it to the end, and replace it with the end number.

Step 1) 2, 6, 5, 0, 8, 7, 1, 3.

Now we see from right → first number greater than 3.

And we see from left → first number lesser than 3.

∴ These are interchanged. (6 & 1)

⇒ 2, 1, 5, 0, 8, 7, 6, 3.

again (5 & 0 are interchanged).

⇒ 2, 1, 0, 5, 8, 7, 6, 3

Now, we stop when

no other numbers are interchangable.

Then, we again put '3' in its original position index.

⇒ 2, 1, 0, 3, 8, 7, 6, 5.

Here, all no. left of 3 ⇒ smaller than 3.

all no. right than 3 ⇒ larger than 3.

Now, the array is divided into 2 parts:

2, 1, 0 | 8, 7, 6, 5.

Here, any pivot; suppose '1'.

then

⇒ 2, 0, 1 (1 & 0 are interchanged)

⇒ 0, 2, 1

⇒ 0, 1, 2. [sorted].

pivot: 7.

⇒ 8, 5, 6, 7.

⇒ 5, 8, 6, 7

⇒ 6, 5, 6, 8, 7.

⇒ 5, 6, 8, 6

5, 6 | 8, 7

5, 6 | 7, 8.

6, 5, 8, 7.

(8 & 6)

= 6, 5, 7, 8

6, 5, 7, 8.

ex: 10, 15, 1, 2, 9, 16, 11.
Let pivot : 10
∴ ⇒ 10, 9, 1, 2, 15, 16, 11

ex: 54, 26, 93, 17, 77, 31, 44, 55, 20.
taking pivot = 54.
⇒ 54, 26, 20, 17, 77, 31, 44, 55, 93.
[20 & 93 interchanged]
⇒ 54, 26, 20, 17, 44, 31, 77, 55, 93
[77 & 44 interchanged]
Nothing more to swap;
∴ '54' is swapped with 31.
⇒ 31, 26, 20, 17, 44, 54, 77, 55, 93.
Now, we have:
31, 26, 20, 17, 44 | 77, 55, 93.
Let pivot : 20 | Let pivot : 77
do, 20 is put to last | ⇒ 77, 93, 55
31, 26, 44, 17, 20. | ⇒ 55, 77, 93.
⇒ 17, 26, 44, 31, 20.
⇒ 17, 20, 44, 31, 26.
17 | 44, 31, 26.
p: 31
44, 26, 31
⇒ 26, 44, 31
⇒ 26, 31, 44.
∴ 17, 20, 26, 31, 44, 54, 55, 77, 93

Time Complexity: Avg, Best: O(n log n)
Worst: O(n²)
Recurrance Relation: T(n) = 2T(n/2) + O(n)

#ALGORITHM:

Input: An array of elements

Output: The sorted array.

1. If the array has 0 or 1 element, it is already sorted return it.
2. Choose a pivot element from the array (ex: last element)
3. Create two new lists:
   a) left: elements less than pivot
   b) right: elements greater than or equal to pivot.
4. Recursively apply Quick sort to the left & right lists.
5. Combine:
6. Return quicksort(left) + pivot + quicksort(right)

2.6 Bubble Sort.

Sorting the array step by step as soon as smaller is seen.

ex: 5, 6, 1, 3

5, 6 → No

6, 1 → Yes

So, 5, 1, 6, 3

6, 3 → Yes,

So, 5, 1, 3, 6.

Again 5, 1 → Yes.

So, 1, 5, 3, 6.

5, 3 → Yes,

So, 1, 3, 5, 6. (Ans).

TIME COMPLEXITY: Best: O(n)

Avg, Worst: $O(n^2)$

#ALGORITHM:

Input: A list of elements

Output: The list sorted in ascending order.

1. Repeat the following steps for each element in the list:
   a. Start from the first element.
   b. Compare each pair of adjacent elements.
   c. If the first is greater than the second, swap them.
2. Keep repeating this until you go through the list without making any changes.

2.7 Insertion Sort.

⇒ Iteratively puts the element in its correct position.

ex: 12, 11, 13, 5, 6.

⇒ 11, 12, 13, 5, 6.

⇒ 11, 12, 13, 5, 6.

⇒ 5, 11, 12, 13, 6.

⇒ 5, 6, 11, 12, 13.

ex: 23, 1, 10, 5, 2

⇒ 1, 23, 10, 5, 2.

⇒ 1, 10, 23, 5, 2

⇒ 1, 5, 10, 23, 2.

⇒ 1, 2, 5, 10, 23.

#ALGORITHM:

Input: A list of elements

Output: The list sorted in ascending order.

1. Start with the 2nd element (index 1) ~~assume the first key is already sorted.~~
2. ~~For~~ each element from index 1 to the end of the list,
   ~~a. Store the current~~
   a. Check whether the next key is greater or smaller.
   b. If smaller, it is put in the correct position in the left side of the Key(current).
   c. If greater, it is left on the right side.
3. The sorted array is achieved.

TIME COMPLEXITY: best: O(n)

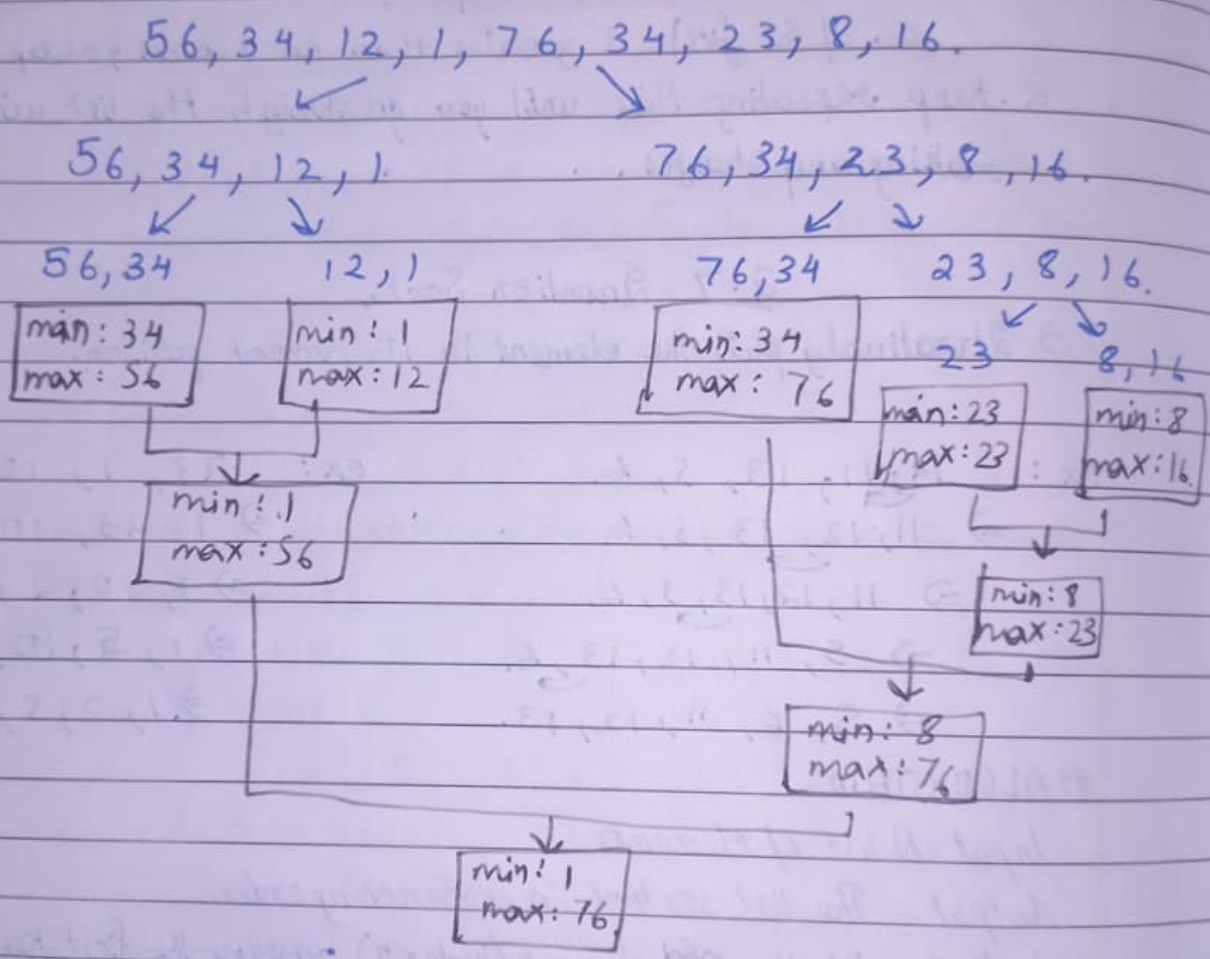
Avg, Worst: O(n²)

2.8 max & min By Divide & Conquer.

→ finding max & min after dividing the given array in subplots.

Given: 56, 34, 12, 1, 76, 34, 23, 8, 16.

Dividing into 2.



∴ min of array = 1

max of array = 76

Time Complexity: best: O(1)

avg, worst: O(n)

Recurrance Relation: $2T(n/2) + 2$.

#ALGORITHM:

1. If the array has only one element:
   - return the element as both max and min.
2. If the array has 2 elements:
   - compare them
   - return the larger as max and the smaller as min.
3. Else:
   - Split the array into 2 halves: left and right.
   - Recursively find the max & min of left half.
   - Recursively find the max & min of right half.
   - Compare the two max values → get overall max.
   - Compare the two min values → get overall min.
   - Return (max, min).

2.9 Closest Pair of Points:
Divide & Conquer:

We are given an array of n points in the plane, and the problem is to find out the closest pair of points in the array.

We know,

The distance between 2 points p and q is:

$$|pq| = \sqrt{(p_x - q_x)^2 + (p_y - q_y)^2}$$

Here,

p_x and p_y are the x and y co-ordinates of point 'p'.

and, q_x and q_y are the x and y co-ordinates of point q.

#Algorithm:

Input: An array of n points P[].

Output: The smallest distance between two points in the given array.

~~Step 1 : Sorting Sorting the array according to the x coordinate.~~

Given array : $P[] = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17\}$

[Here basically a given graph is these].

Step 1) Sorting the array according to the x-coordinate :
seeing the graph,

$P[] = \{13, 11, 12, 0, 14, 16, 1, 10, 17, 9, 2, 15, 3, 8, 4, 5, 7, 6\}$

Step 2) Finding the middle point : i.e = 17.

$P[n/2] = P[18/2] = P[9] = 17.$

Step 3) Dividing the array into 2 parts :

$P_L[] = \{13, 11, 12, 0, 14, 16, 1, 10, 17\}$ | $P_R[] = \{9, 2, 15, 3, 8, 4, 5, 7, 6\}$

[we slipt the graph into 2 parts].

Let dl be the smallest distance in the array P_L.
and dr be the smallest dist. in the array P_R.

2.10 Convex Hull :

→ A polygon is convex if any line segment joining two points on the boundary stays within the polygon.

→ The convex hull of a set of points in the plane is the smallest convex polygon for which each point is either on the boundary or in the interior of the polygon.

→ A vertex is a corner of a polygon. for ex, the highest, lowest, leftmost, rightmost points are all vertices of the convex hull.

Can be done by:

→ Graham Scan

→ Jarvis March

→ Divide & Conquer.

(1) Divide & Conquer:

Suppose a given graph is given.

Step 1) We divide it approximately into 2 halves. [by X co-ordinate]

Step 2) We draw the polygon connected ~~to~~ points such that biggest area is found on both sides.

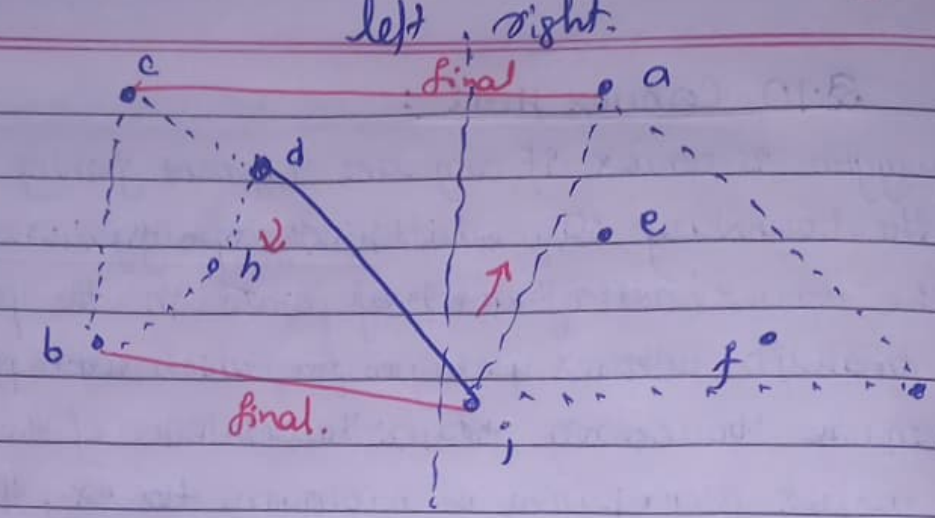
~~Step 3) We the~~

(1) Now, At the ~~left~~ left side, we notice the right most point, and in the right side, we notice the leftmost point.

(2) We connect them, if it creates the lowest bound, wall and fine, otherwise we go next point of the chosen "leftest point" clockwise, unless LOWEST BOUND is found.
[clockwise for left, anticlockwise for right].

(3) Similarly, for the ~~right~~ UPPER BOUND; we ~~choose the~~ connect ~~to~~ the right most of the ~~left~~ left, and leftmost of the right, unless upper bound is got.
[anticlockwise for left, clockwise for right].

Ex:



Here,

Step 1) right most of left : d

left most of right : j.

but this is not the lowest bound.

∴ by clockwise direction, it goes to 'b'.

even 'b' is not the lowest bound.

[finally after all clockwise movements of left & all anticlockwise movement of right].

we get lower bound : b j

Step 2) Similarly,

[finally after all anticlockwise from left & clockwise from right].

we get upper bound : c a.

∴ Total Convex Hull : b j g a c b

Time Complexity : $O(n \log n)$

recurrance relation : $T(n) = 2T(n/2) + O(n)$

#Algorithm:

Input: a set of points on a 2D plane.

output: a list of points that form the convex hull (the outer boundary).

1. Sort the points by x-coordinates. (and y to break ties)
2. Divide the points into 2 halves:
   - Left half and Right half.
3. Recursively compute the convex hull for the left half.
4. Recursively compute the convex hull for the right half.
5. Merge the 2 convex hulls.
   (i) Find the upper tangent & lower tangent between the two hulls.
   (ii) Remove the points between the tangents that are inside.
   (iii) Combine the points from both hulls that form the outer boundary.
6. Return the merged convex hull.

—x—